MODUL PRAKTIKUM PEMROGRAMAN TERSTRUKTUR

Pertemuan 1: Dari Python ke C++: Tools, Program Pertama, dan Literasi AI

Mata Kuliah	Pemrograman Terstruktur (TIF1302), 3 SKS (T 3, P 0), semester 3
Pertemuan	1 dari 16
Sub-CPMK	**CPMK032.1** Mampu membangun program C++ dasar: tipe data, variabel, konstanta, ekspresi, dan operator, sesuai kaidah bahasa C++.
Indikator	Menjelaskan tahap preprocess, compile, link, dan run; membangun program C++ pertama tanpa error dan tanpa warning; mengenali empat jenis kesalahan program (sintaks, compile, runtime, logika); dan menjelaskan dengan contoh konkret batas pemakaian AI pada tugas kuliah.
Penilaian	Tes diagnostik (tanpa bobot) dan praktikum program pertama. Bobot 0 persen.
Alokasi Waktu	Tatap muka 3 x 50 menit; penugasan terstruktur 3 x 60 menit; belajar mandiri 3 x 60 menit
Perangkat	MSYS2 UCRT64 dengan GCC (termasuk g++) dan GDB, Visual Studio Code dengan ekstensi C/C++ dan Code Runner, Git, Python 3, serta akun GitHub untuk repository `pt-NPM`.
Prasyarat	Algoritma dan Pemrograman 2. Sudah menyelesaikan Panduan Persiapan Perangkat dan swa-uji pada slide Persiapan Pertemuan 1.

A. Tujuan Praktikum

1. Menceritakan dengan kata-kata sendiri apa yang terjadi sejak kode C++ ditulis sampai menjadi program yang bisa dijalankan.
2. Membandingkan cara kerja Python dan C++, lalu menjelaskan apa akibatnya bagi kita yang menulis kodenya.
3. Membangun dan menjalankan program C++ sendiri, dari terminal maupun dari tombol di VS Code, sampai tidak ada error dan tidak ada warning yang tersisa.
4. Membedakan empat jenis kesalahan program dari gejalanya, dan tahu siapa yang memberi tahu kita untuk setiap jenisnya.
5. Menyimpan hasil kerja hari ini ke repository pribadi di Git dan GitHub, lalu menuliskan deklarasi AI dengan jujur.

B. Ringkasan Teori

B.1 Python Diterjemahkan Sambil Jalan, C++ Diterjemahkan Semua Dulu

Bayangkan dua penerjemah. Penerjemah pertama membacakan sebuah cerita sambil langsung menerjemahkan kalimat demi kalimat ke pendengarnya. Begitu ada satu kalimat aneh di tengah cerita, dia baru bingung setelah

sampai di kalimat itu, padahal bagian sebelumnya sudah terlanjur didengar. Penerjemah kedua menerjemahkan seluruh cerita dulu dari awal sampai akhir, baru menyerahkan hasilnya. Kalau ada satu kalimat saja yang tidak bisa diterjemahkan, cerita itu tidak akan pernah selesai sampai kalimat tersebut diperbaiki.

Python bekerja seperti penerjemah pertama. Kode `.py` dibaca dan dijalankan baris demi baris oleh interpreter, saat itu juga. Karena itu, kesalahan seperti salah ketik nama variabel baru ketahuan ketika baris tersebut benar-benar dijalankan, kadang setelah program berjalan cukup lama.

C++ bekerja seperti penerjemah kedua. Sebelum program bisa dijalankan sama sekali, seluruh kode `.cpp` harus selesai diterjemahkan lebih dulu menjadi berkas program, misalnya `hello.exe`. Proses ini disebut compile, dan alat yang mengerjakannya disebut compiler. Di kelas ini compiler kita bernama `g++`. Semua kesalahan penulisan wajib beres duluan, sebab kalau tidak, berkas programnya tidak akan pernah terbentuk.

Yang dibandingkan	Python	C++
Kapan salah ketik nama variabel ketahuan	Saat baris itu dijalankan	Saat kode diterjemahkan, program gagal dibuat
Bentuk akhirnya	Tetap berupa berkas `.py`	Jadi berkas program, misalnya `hello.exe`
Tipe variabel	Ditebak Python dari isinya	Kita tulis sendiri: `int`, `double`, dan seterusnya
Akhir sebuah perintah	Ganti baris	Tanda titik koma `;`
Mencetak ke layar	`print("Halo")`	`std::cout << "Halo\n";`
Soal kecepatan	Lebih lambat, diterjemahkan sambil jalan	Lebih cepat, sudah jadi kode mesin

Catatan: Ini bukan soal bahasa mana yang lebih unggul. Kita belajar C++ karena C++ memaksa kita menuliskan hal-hal yang selama ini ditebak diam-diam oleh Python, misalnya tipe setiap variabel. Kebiasaan itu yang nanti dipakai lagi di Pemrograman Berbasis Objek.

B.2 Empat Tahap dari Kode Sampai Menjadi Program yang Bisa Dijalankan

Tahap	Apa yang terjadi	Contoh pesan kalau tahap ini gagal
1. Preprocess	Menyalin isi berkas yang di-`#include`, misalnya `<iostream>`	`fatal error: iostraem: No such file or directory`
2. Compile	Kode C++ diperiksa ejaan dan tipenya, lalu diubah menjadi kode objek	`error: expected ',' or ';' before 'std'`
3. Link	Semua kode objek dan pustaka digabung menjadi satu program utuh	undefined reference to WinMain (fungsi `main` tidak ditemukan)
4. Run	Program dijalankan oleh sistem operasi	Program berhenti mendadak, atau hasilnya salah

Satu perintah `g++` menjalankan tiga tahap pertama sekaligus, satu demi satu, lalu kita sendiri yang menjalankan hasilnya. Begitu muncul pesan error, pertanyaan pertama yang paling berguna adalah: tahap mana yang gagal? Sebab penyebab dan cara memperbaikinya berbeda-beda untuk tiap tahap.

B.3 Perintah Build yang Kita Pakai Sepanjang Semester

Supaya hasilnya konsisten buat semua orang, seluruh program di mata kuliah ini dibangun dengan satu perintah yang sama. Perintah ini sengaja menyalakan beberapa pemeriksaan tambahan, supaya kalau ada yang salah, itu langsung kelihatan sebagai pesan yang bisa dibaca, bukan program yang diam-diam keliru.

Perintah build kelas

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -g hello.cpp -o hello
```

Bagian perintah	Fungsinya, yang penting dipahami sekarang
`-std=c++20`	Supaya compiler semua orang memakai aturan bahasa C++ yang persis sama, yaitu C++20
`-Wall -Wextra -Wpedantic`	Menyalakan peringatan (warning) untuk kode yang mencurigakan, misalnya variabel yang dibuat tapi tidak pernah dipakai
`-g`	Menyimpan informasi tambahan supaya debugger bisa menunjuk baris kode aslinya. Debugger baru kita pakai pada Pertemuan 6
`hello.cpp -o hello`	Berkas yang diterjemahkan, dan nama program hasilnya

Tombol di VS Code: Kamu tidak perlu mengetik perintah itu setiap kali. Folder kode pertemuan ini sudah memuat pengaturan VS Code: tekan **Ctrl+Shift+B** untuk build saja, atau klik tombol Run Code dari Code Runner untuk build lalu langsung jalan. Keduanya memakai perintah yang persis sama. Pada pertemuan awal, sesekali ketik sendiri perintahnya di terminal supaya tahap-tahap di B.2 tetap terasa.

Aturan kelas: Target kelas adalah build tanpa error dan tanpa warning. Error memang menghentikan segalanya, sedangkan warning masih menghasilkan program, tetapi hampir selalu menunjuk sesuatu yang belum selesai atau keliru. Warning yang tersisa kita bahas dan perbaiki bersama, bukan didiamkan.

B.4 Membaca Program C++ Pertama Kita Baris demi Baris

hello.cpp

```
#include <iostream>

int main() {
    std::cout << "Halo dari C++\n";
    return 0;
}
```

Baris	Artinya
`#include <iostream>`	Meminjam perkakas input dan output dari pustaka standar. Tanpa baris ini, `std::cout` tidak dikenal
`int main() {`	Pintu masuk program. Setiap program C++ punya tepat satu `main`, dan dari sinilah eksekusi dimulai
`std::cout << "Halo dari C++\n";`	Kirim teks ke layar. `std::cout` adalah layarnya, `<<` artinya "kirim ke", dan `\n` adalah ganti baris

Baris	Artinya
`return 0;`	Beri tahu sistem operasi bahwa program selesai dengan baik. Angka selain 0 berarti ada masalah
`}`	Penutup `main`. Setiap `{` selalu berpasangan dengan `}`

B.4.1 Apa Sebenarnya std:: Itu

Bayangkan di kelasmu ada dua orang bernama Budi. Supaya tidak tertukar, orang memanggil mereka dengan marganya: Budi Santoso dan Budi Wijaya. C++ punya masalah yang sama: ada ribuan nama perkakas, dan bisa saja dua perkakas dari sumber berbeda punya nama yang sama. Solusinya, setiap kumpulan perkakas diberi "marga" yang disebut namespace, dan marga untuk seluruh perkakas bawaan C++ adalah `std` (singkatan dari standard).

Jadi `std::cout` dibaca "cout dari keluarga std". Tanda `::` adalah pemisah antara marga dan nama. Semua yang kamu pinjam lewat `#include <iostream>` atau `#include <string>` bermarga `std`: `std::cout`, `std::cin`, `std::string`, `std::getline`, dan seterusnya. Kalau marganya tidak ditulis, compiler tidak menemukan namanya dan menolak dengan pesan seperti `'cout' was not declared in this scope`.

Jalan pintas yang akan kamu temui: Di internet dan buku kamu akan sering melihat kode yang menulis `cout` saja tanpa `std::`. Kode seperti itu memakai jalan pintas `using namespace std;` yang artinya "kalau ada nama yang tidak ditemukan, cari di keluarga std". Hari ini kita sengaja menulis marganya lengkap supaya kamu tahu dari mana setiap nama berasal. Jalan pintasnya diperkenalkan pada Pertemuan 2, dan sejak itu kode kelas memakainya.

B.5 Empat Jenis Kesalahan, dan Siapa yang Memberi Tahu Kita

Tidak semua kesalahan sama. Ada yang langsung ditolak compiler, ada yang lolos sampai program berjalan, dan ada yang tidak pernah dilaporkan siapa pun. Mengenali jenisnya membuat kita tahu ke mana harus mencari.

Jenis	Gejalanya	Siapa yang memberi tahu	Contoh di folder kode
Sintaks	Build gagal, biasanya karena titik koma, tanda kutip, atau kurung yang hilang	Compiler, saat compile, lengkap dengan nomor baris	`k1_sintaks.cpp`: titik koma hilang
Compile lainnya	Build gagal karena nama belum dikenal atau tipe tidak cocok	Compiler, sering sekaligus menebak yang kamu maksud	`k2_nama.cpp`: `Nilai` bukan `nilai`, `bonus` belum dibuat
Runtime	Build sukses, tetapi program berhenti mendadak saat dijalankan dengan masukan tertentu	Sistem operasi, tanpa penjelasan yang ramah	`k3_runtime.cpp`: dibagi nol saat pengguna mengetik 0
Logika	Build sukses, program jalan mulus, hasilnya salah	Tidak ada. Hanya ketahuan kalau kita mencoba dan membandingkan hasilnya	`k4_logika.cpp`: rata-rata jadi 81, bukan 81,67

Perhatikan: Kesalahan logika yang paling perlu diwaspadai, justru karena tidak ada yang memberi peringatan. Compiler dan warning banyak membantu, tetapi program yang sudah bebas warning belum tentu benar. Menguji hasilnya tetap jadi tanggung jawab kita sendiri.

B.6 Boleh Pakai AI, Tapi Ada Aturannya

AI seperti ChatGPT atau Copilot bisa sangat membantu, dan juga bisa membuat kita tidak belajar apa-apa. Bedanya ada pada cara memakainya. Aturan kelas ini sederhana: AI boleh membantu kamu memahami, tidak boleh menggantikan kamu mengerjakan. Setiap tugas diberi label yang menyatakan sejauh mana AI boleh dipakai.

Label	Ketentuan	Contoh yang boleh
AI 0	AI tidak diperbolehkan sama sekali	Tes diagnostik, kuis, UTS, tugas besar, dan seluruh praktikum hari ini
AI 1	AI hanya untuk penjelasan atau kritik setelah kamu berusaha dulu sendiri	Menempelkan satu pesan error dan bertanya artinya; meminta kritik atas rancangan yang sudah kamu buat
AI 2	AI boleh membantu code review dan mencari bug, disertai bukti kamu memeriksanya	Meminta AI membaca kodemu yang sudah jadi dan menunjukkan bagian yang mencurigakan, lalu kamu uji sendiri

Boleh, pada tugas berlabel AI 1	Tidak boleh, apa pun labelnya
"Apa arti pesan error ini: expected ; before std?"	"Buatkan program C++ yang menghitung rata-rata tiga nilai."
"Apa beda 7 / 2 dan 7 / 2.0 di C++?"	"Kerjakan TODO di berkas ini."
"Saya sudah menulis kode ini. Menurutmu bagian mana yang mungkin salah?"	Menempelkan soal ujian atau data pribadi teman ke layanan AI

Setiap tugas yang dikumpulkan disertai deklarasi AI: labelnya, alat yang dipakai (kalau ada), untuk apa, dan bagaimana kamu memeriksa kebenarannya. Kalau tidak memakai AI, tulis saja "tidak memakai AI". Satu aturan yang tidak bisa ditawar: kode yang tidak bisa kamu jelaskan sendiri tidak dinilai, apa pun labelnya.

Hari ini: Seluruh praktikum hari ini berlabel AI 0, sengaja. Kamu perlu merasakan sendiri membaca pesan compiler dan menemukan kesalahan tanpa bantuan, supaya mulai Pertemuan 2 (label AI 1) kamu tahu kapan AI benar-benar membantu dan kapan hanya membuatmu melewatkan pelajaran.

B.7 Kesalahan yang Sering Terjadi di Pertemuan Pertama

Gejalanya	Penyebab dan cara mengatasinya
`g++: command not found`	Terminalnya bukan Git Bash yang sudah diatur sesuai panduan persiapan, atau PATH belum diatur. Tutup terminal, buka lagi dari VS Code, atau pakai terminal MSYS2 UCRT64.
Semua pesan compiler di PowerShell tampak merah	PowerShell menganggap semua keluaran `g++` sebagai error, padahal belum tentu. Pakai Git Bash saja.
undefined reference to WinMain	Fungsi `main` tidak ditemukan, biasanya salah ketik seperti `Main` atau `mian`. C++ membedakan huruf besar dan kecil.
`'cout' is not a member of 'std'`	Baris `#include <iostream>` terlupa atau salah ketik.
Program lama masih jalan padahal kode sudah diubah	Build-nya sebenarnya gagal, jadi berkas `.exe` lama yang masih terpakai. Baca dulu pesan errornya dari baris paling atas.
`warning: unused variable`	Ada variabel yang dibuat tapi tidak dipakai. Pada lab porting, ini tanda TODO-nya belum dikerjakan.

C. Langkah Praktikum

Semua berkas untuk praktikum ini ada di folder `04_Kode/P01_Program_Pertama`. Buka folder itu di VS Code lewat File, Open Folder, supaya pengaturan `.vscode`-nya ikut terpakai. Kerjakan urut dari atas, dan jalankan semua perintah dari terminal Git Bash. Hari ini berlabel **AI 0**.

Praktikum 1: Memastikan Semuanya Sudah Siap

Langkah	Aktivitas
1	Buka terminal di VS Code dengan Ctrl+` dan pastikan yang terbuka adalah Git Bash, bukan PowerShell.
2	Cek versi compiler dengan `g++ --version`. Nomor versinya harus muncul tanpa pesan error.
3	Buka `hello.cpp`, lalu bangun dari terminal dengan perintah di bawah. Berhasil bila tidak ada pesan apa pun, dan `./hello` mencetak `Halo dari C++`.
4	Sekarang coba cara kedua: tekan **Ctrl+Shift+B**. Panel terminal menampilkan perintah yang sama dijalankan otomatis.
5	Cara ketiga: klik tombol Run Code (segitiga di pojok kanan atas editor, atau Ctrl+Alt+N). Program dibangun lalu langsung dijalankan.

Perintah build dan jalankan

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -g hello.cpp -o hello
./hello
```

Praktikum 2: Membuktikan Standar Bahasanya

Program kedua ini mencetak nomor versi standar C++ yang dipakai compiler. Bangun dan jalankan dengan cara mana saja yang kamu suka dari Praktikum 1.

versi.cpp

```
// Program kedua: membuktikan compiler memakai standar C++20.
// Nilai __cplusplus untuk C++20 adalah 202002.
#include <iostream>

int main() {
    std::cout << "Halo, Pemrograman Terstruktur\n";
    std::cout << "Standar bahasa yang dipakai compiler: " <<   cplusplus << "\n";
    return 0;
}
```

Baris kedua keluarannya harus memuat angka `202002`, tandanya compiler memakai C++20. Kalau angka yang muncul beda, laporkan ke dosen, karena berarti opsi `-std=c++20` belum terpakai dengan benar.

Praktikum 3: Bandingkan Dulu dengan Versi Python

Berikut ini program Python yang nanti akan kita pindahkan ke C++. Jalankan dulu, supaya kita tahu hasil yang benar itu seperti apa.

rerata.py

```
# Versi Python: rata-rata tiga nilai. Program ini yang akan dipindahkan ke C++.
tugas = 80
uts = 75
uas = 90

jumlah = tugas + uts + uas
rerata = jumlah / 3

print("Jumlah    :", jumlah)
print(f"Rata-rata : {rerata:.2f}")
```

Menjalankan versi Python

```
python rerata.py
```

Catat hasilnya: jumlahnya 245 dan rata-ratanya 81,67. Python langsung menghasilkan pecahan dari `/` tanpa diminta, dan `:.2f` membulatkannya ke dua angka di belakang koma.

Praktikum 4: Sekarang Pindahkan ke C++

Buka `rerata_awal.cpp`, lalu lengkapi ketiga bagian TODO. Ingat dua hal: di C++ tipe data harus ditulis, dan pembagian antara dua bilangan bulat membuang bagian pecahannya. Kalau kamu membangunnya sebelum mengisi TODO, muncul tiga warning `unused variable`. Itu bukan kesalahan berkasnya, itu compiler mengingatkan bahwa `tugas`, `uts`, dan `uas` belum kamu pakai.

rerata_awal.cpp

```
// Lab porting: pindahkan rerata.py ke C++.
// Lengkapi tiga bagian bertanda TODO, lalu bangun dengan baseline kelas.
#include <iomanip>
#include <iostream>

int main() {
    int tugas = 80;
    int uts = 75;
    int uas = 90;

    // TODO 1: hitung jumlah ketiga nilai. Di C++ tipe variabel wajib ditulis.
    int jumlah = 0;

    // TODO 2: hitung rata-rata. Ingat, int dibagi int membuang pecahannya.
    //         Pakai tipe double dan pastikan pembagiannya bukan pembagian bilangan bulat.
    double rerata = 0;

    // TODO 3: cetak hasil dengan dua angka di belakang koma, sama seperti versi Python.
    std::cout << "Jumlah    : " << jumlah << "\n";
    std::cout << "Rata-rata : " << rerata << "\n";
    return 0;
}
```

Perintah build dan jalankan

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -g rerata_awal.cpp -o rerata
./rerata
```

Hasil yang benar sama persis dengan versi Python: `Jumlah    : 245` dan `Rata-rata : 81.67`. Untuk TODO 3, dua baris berikut membuat `std::cout` mencetak tepat dua angka di belakang koma. Cukup salin dulu; `<iomanip>` dan kawan-kawannya kita bahas pada Pertemuan 6.

Petunjuk TODO 3

```
std::cout << std::fixed << std::setprecision(2);
std::cout << "Rata-rata : " << rerata << "\n";
```

Setelah keluarannya benar dan build-nya bersih tanpa warning, simpan berkasnya dengan nama `rerata.cpp`. Lalu bandingkan: berapa keputusan tambahan yang harus kamu ambil di versi C++ dibanding versi Python tadi? Catat di lembar refleksi.

Praktikum 5: Katalog Kesalahan, Empat Berkas Empat Jenis

Folder `kesalahan` berisi empat program yang sengaja dibuat salah, satu untuk setiap jenis kesalahan di B.5. Bangun satu per satu dengan baseline kelas, baca pesan yang muncul (atau tidak muncul), lalu isi tabel di bawah pada berkas `catatan_kesalahan.md`.

kesalahan/k1_sintaks.cpp

```
// Kesalahan 1: sintaks. Ada satu tanda titik koma yang hilang.
// Program ini gagal pada tahap compile, berkas .exe tidak terbentuk.
#include <iostream>

int main() {
    int nilai = 80
    std::cout << "Nilai: " << nilai << "\n";
    return 0;
}
```

kesalahan/k2_nama.cpp

```
// Kesalahan 2: nama yang belum dikenal. Variabel dipakai sebelum dideklarasikan,
// dan ada salah ketik huruf besar. Di Python kesalahan ini baru terasa saat baris
// itu dijalankan; di C++ ditolak compiler sebelum program pernah berjalan.
#include <iostream>

int main() {
    int nilai = 80;
    std::cout << "Nilai: " << Nilai << "\n";
    std::cout << "Bonus: " << bonus << "\n";
    return 0;
}
```

kesalahan/k3_runtime.cpp

```
// Kesalahan 3: runtime. Kode ini lolos compile tanpa error dan tanpa warning,
// tetapi berhenti mendadak saat pengguna memasukkan 0 sebagai jumlah mahasiswa.
#include <iostream>

int main() {
    int total = 240;
    int jumlah_mahasiswa = 0;
    std::cout << "Jumlah mahasiswa: ";
    std::cin >> jumlah_mahasiswa;
    int rerata = total / jumlah_mahasiswa;
    std::cout << "Rata-rata: " << rerata << "\n";
    return 0;
}
```

kesalahan/k4_logika.cpp

```
// Kesalahan 4: logika. Program berjalan mulus, tidak ada pesan apa pun,
// tetapi hasilnya salah. Rata-rata 80, 75, dan 90 seharusnya 81.67, bukan 81.
#include <iostream>

int main() {
    int tugas = 80;
    int uts = 75;
    int uas = 90;
    double rerata = (tugas + uts + uas) / 3;
    std::cout << "Rata-rata: " << rerata << "\n";
    return 0;
}
```

Berkas	Jenis kesalahan	Pesan yang muncul (salin baris pertamanya)	Cara kamu mengetahuinya
`k1_sintaks.cpp`			
`k2_nama.cpp`			
`k3_runtime.cpp`			
`k4_logika.cpp`			

Langkah	Aktivitas
1	Untuk `k1` dan `k2`, baca pesan compiler dari baris paling atas. Perhatikan nomor baris dan kolomnya, lalu perbaiki sampai build bersih.
2	Untuk `k3`, build-nya sukses. Jalankan dua kali: ketik `4` lalu ketik `0`. Amati bedanya. Cara mencegahnya butuh percabangan, jadi cukup dicatat dulu, kita perbaiki pada Pertemuan 4.
3	Untuk `k4`, build-nya sukses dan program jalan mulus. Bandingkan hasilnya dengan Praktikum 3, temukan sebabnya, lalu perbaiki dengan satu perubahan kecil.

Cara membaca pesan compiler: Pesan compiler memang panjang dan tampak menakutkan, tetapi polanya selalu sama: `nama_berkas:baris:kolom: jenis: penjelasan`. Baca bagian nama berkas dan nomor baris dulu, baru penjelasannya. Compiler modern bahkan sering menebak yang kamu maksud, misalnya "did you mean nilai?".

Praktikum 6: Simpan Pekerjaan ke Git dan GitHub

Mulai hari ini semua kode kuliah disimpan di satu repository pribadi bernama `pt-NPM`, dengan satu folder untuk setiap pertemuan. Ganti NPM dengan NPM kamu sendiri.

Langkah	Aktivitas
1	Di luar folder kode kuliah, buat folder `pt-NPM` (misalnya `pt-2024010101`), lalu di dalamnya buat folder `p01`.
2	Salin `hello.cpp`, `rerata.cpp` hasil porting, dan `catatan_kesalahan.md` ke folder `p01`.
3	Salin berkas `.gitignore` dari folder kode ke folder `pt-NPM` supaya hasil build tidak ikut tersimpan.

Langkah	Aktivitas
4	Buka Git Bash di folder `pt-NPM`, lalu jalankan perintah pada kotak di bawah.
5	Buat repository kosong di GitHub dengan nama yang sama, `pt-NPM`, tanpa README. Salin dua perintah yang ditawarkan GitHub (`git remote add origin` dan `git push -u origin main`) lalu jalankan.
6	Buka halaman repository di browser, pastikan folder `p01` terlihat, lalu kumpulkan tautannya lewat LMS.

Git Bash, di dalam folder pt-NPM

```
git init
git add .
git commit -m "Pertemuan 1: program pertama"
```

Login GitHub: Kalau Git meminta login saat push, pilih Sign in with your browser lalu izinkan akses. GitHub tidak menerima password akun untuk perintah Git.

D. Latihan Mandiri

1. Ubah `rerata.cpp` supaya menghitung rata-rata dari lima nilai. Perhatikan bagian mana saja yang perlu diubah, dan catat berapa tempat yang terpengaruh.
2. Hapus tanda kutip penutup pada `hello.cpp`, bangun ulang, lalu salin pesan error yang muncul beserta nomor barisnya. Kembalikan lagi seperti semula setelah itu.
3. Hapus baris `#include <iostream>` di `hello.cpp`, bangun ulang, lalu jelaskan tahap mana yang gagal dan kenapa pesannya beda dengan latihan sebelumnya.
4. Bangun `rerata_awal.cpp` yang masih kosong tanpa opsi `-Wall -Wextra`, lalu bandingkan: pesan apa yang hilang, dan kenapa itu justru merugikan?

E. Tugas yang Dikumpulkan

No	Yang dikumpulkan	Ketentuan
1	Tautan repository `pt-NPM`	Folder `p01` berisi `hello.cpp`, `rerata.cpp` hasil porting, dan `catatan_kesalahan.md`; berkas `.exe` tidak ikut
2	Berkas `catatan_kesalahan.md`	Tabel Praktikum 5 yang sudah terisi untuk keempat berkas, ditambah satu kalimat: kesalahan jenis mana yang menurutmu paling berbahaya dan kenapa
3	Lembar refleksi (LMS)	Tiga perbedaan Python dan C++ yang paling terasa, ditulis dengan kalimat sendiri
4	Deklarasi AI (LMS)	Untuk hari ini seharusnya berbunyi "AI 0, tidak memakai AI". Tulis apa adanya kalau kenyataannya berbeda

Batas waktu pengumpulan diumumkan lewat LMS. Semua berkas `.cpp` harus bisa dibangun dengan baseline kelas tanpa error dan tanpa warning.

F. Umpan Balik dan Kriteria

Pertemuan ini tidak memiliki bobot nilai (lihat kontrak kuliah): tes diagnostik dan praktikum hari ini bersifat formatif. Umpan balik diberikan lewat daftar periksa berikut, dan repository yang terkumpul hari ini menjadi tempat seluruh tugas berbobot mulai Pertemuan 2.

Aspek	Kriteria tercapai
Lingkungan dan build	hello.cpp dan versi.cpp dibangun tanpa error dan tanpa warning; versi mencetak 202002
Hasil porting	Keluaran `rerata.cpp` sama persis dengan versi Python, sampai dua angka di belakang koma
Katalog kesalahan	Keempat berkas dikenali jenis kesalahannya dengan tepat, k1, k2, dan k4 diperbaiki
Git dan integritas	Repository terkumpul dengan `.gitignore` benar, pesan commit bermakna, refleksi dan deklarasi AI terisi

G. Pertanyaan Refleksi

1. Kenapa salah ketik nama variabel di C++ tidak pernah "lolos" sampai ke tangan pengguna program, sedangkan di Python bisa?
2. Apa sebenarnya yang hilang saat kita membagi dua bilangan bulat di C++? Kapan hal ini bisa berbahaya pada program yang benar-benar dipakai orang?
3. Dari empat jenis kesalahan hari ini, yang mana yang paling mudah ditemukan dan yang mana yang paling sulit? Apa yang membuatnya berbeda?
4. Kalau kamu menempelkan `rerata_awal.cpp` ke AI dan memintanya mengisi TODO, apa yang kamu dapat dan apa yang kamu lewatkan?

H. Referensi

- [U1] Stroustrup, B. Programming: Principles and Practice Using C++. Edisi kedua. Bab 2, Hello, World!
- [U3] ISO/IEC 14882:2020. Programming Languages - C++, bagian 5.2 tentang tahapan penerjemahan.
- [P1] GCC Documentation, bagian Options to Request or Suppress Warnings.
- [P2] cppreference.com, halaman std::cout dan std::setprecision.
- Panduan Persiapan Perangkat Pemrograman Terstruktur, bagian 1 sampai 12, dan Kontrak Perkuliahan bagian 6 tentang kebijakan AI.